

Claude CLI vs Deel CLI

동일 프로젝트 · 서로 다른 데이터 손실 결함 1건씩 탐지/수정 · 2회 실험 결과

CLAUDE CLI

\$7.90 / 38분

claude-token-saver 별도 npm 적용

DEEL CLI

\$6.57 / 36분 59초

별도 축소/조정 없이 자체 기능

비용 차이

-\$1.33

Deel 관측 비용 16.8% 낮음

시간 차이

-61초

Deel 2.7% 빠름

한눈에 보는 결론

품질

둘 다 수정 전 데이터 손실을 재현하고 최소 수정 후 회귀 검증까지 완료. 결과 품질은 대체로 동급.

검증

Claude는 collaboration 50 + design 11 + presets 18까지 실행해 전체 회귀 범위가 더 넓음.

효율

시간은 사실상 동일하지만 Deel이 \$1.33 저렴. 단 2회 표본이므로 일반적 비용 우위로 단정 불가.

비교 조건과 해석 경계

도구 자체 성능과 외부 토큰 절감 효과를 섞어 해석하지 않도록 조건을 분리

Claude CLI

실행 시간 38분 · 비용 \$7.90

별도 npm 패키지 claude-token-saver 사용 조건. 따라서 이 비용은 순정 Claude CLI의 무조정 비용이 아니라 외부 절감 레이어가 포함된 관측값.

Deel CLI

실행 시간 2,219초(36분 59초) · 비용 \$6.57

별도 토큰 축소/조정 패키지 없이 자체 기능으로 수행한 관측값. 이번 결과에서는 Claude보다 낮은 비용을 기록.

공정한 해석

현재 비교는 "실제 사용 구성"끼리의 비교다. Claude는 token-saver가 포함되고 Deel은 자체 기능만 사용했다. 따라서 \$1.33 차이를 Deel 엔진 자체의 순수 효율 차이라고 단정할 수 없다. 순수 CLI 효율을 비교하려면 Claude 순정 / Claude+token-saver / Deel의 3조건 반복 측정이 필요하다.

프롬프트가 결과를 어떻게 바꿨나

제공된 실행 기록에서 확인되는 사용자 지시를 익명화해 구조와 탐지 방식의 차이를 비교

1번 프롬프트 · 탐색형

핵심 지시

공동작업 기능 분석
동시 편집의 데이터 손실·덮어쓰기·충돌·상태 불일치 탐색
실제 코드 근거만 제시
관련 파일·함수·발생 조건·원인 설명
최대 5개
기록상 초기 지시는 코드 수정 금지

프롬프트 성격

“문제를 찾아 설명하라”에 가까운 감사/audit형. 탐색 폭과 근거 제시에 무게가 있다.

2번 프롬프트 · 배포 전 개선형

핵심 지시

구조와 데이터 흐름부터 파악
편집→저장→전파→충돌→재연결→Undo/Redo 추적
영향이 큰 실제 문제 1개 선정
기존 방어 로직 확인·오탐 제거
수정 전 재현 테스트 우선
최소 범위 수정
신규+기존 회귀 테스트
실패 원인 재분석·주변 재검토·미검증 항목 보고

프롬프트 성격

“재현→원인→최소 수정→증명”을 강제한 end-to-end 디버깅형. 한 문제를 끝까지 닫는 데 무게가 있다.

주의 1번 자료 묶음에는 “코드 수정 금지” 초기 지시와 실제 수정·검증 결과가 함께 존재한다. 따라서 아래 비교는 최종 실행 산출물은 관측 사실로, 프롬프트는 기록에서 확인되는 초기 지시로 구분해 표시한다.

무엇을 어떤 식으로 탐지했나

프롬프트 → 코드 탐색 → 이상 징후 → 재현 → 수정/검증의 실제 흐름을 비교

1번 · IME/base 경로

- 1. 탐색 한글 composition 보류 경로 확인
- 2. 비대칭 화면 적용은 보류하지만 base가 먼저 이동
- 3. 증거 delta 기준과 실제 화면 state가 다른 시점
- 4. 재현 B의 새 값이 A의 예전 값으로 되돌아감
- 5. 검증 신규 7개 + 광범위 회귀

2번 · await/TOCTOU 경로

- 1. 탐색 서버 50개 baseline 후 클라이언트 sync 추적
- 2. 비대칭 apply 호출부 중 receive 경로만 rebase 누락
- 3. 증거 await 전 가드만 있고 응답 후 재확인 없음
- 4. 재현 A-TYPED-DURING-AWAIT → B-TITLE로 유실
- 5. 검증 신규 6개 + 50회귀 + mutation

핵심 차이

1번은 특수 상태(IME) 안에서 기준값 관리 불일치를 깊게 파고들었고, 2번은 전체 동기화 흐름을 비교하면서 “다른 경로에는 있는데 한 경로에만 없는 rebase”라는 구조적 비대칭을 찾아 레이스를 특정했다.

서로 다른 결함을 잡았다

두 실행은 같은 버그를 반복 탐지한 것이 아니라 공동작업 동기화의 다른 실패 경로를 각각 제거

1번 · Claude CLI

IME composition / base 오염

- A가 한글 조합 중
- B의 원격 변경 도착
- 화면 적용은 보류
- base만 먼저 최신화
- 다음 delta가 B 변경을 되돌림

핵심: 화면 상태와 delta 기준값이 서로 다른 시점을 가리키며 서버가 정상 편집으로 오인.

2번 · Deel CLI

receive await / TOCTOU race

- 원격 변경 수신 시작
- GET /api/state await
- 그 사이 A가 새 입력
- 응답 후 재확인 없음
- 서버 문서가 A 입력을 덮어씀

핵심: await 이전의 안전성 검사를 응답 이후에도 유효하다고 가정한 TOCTOU 문제.

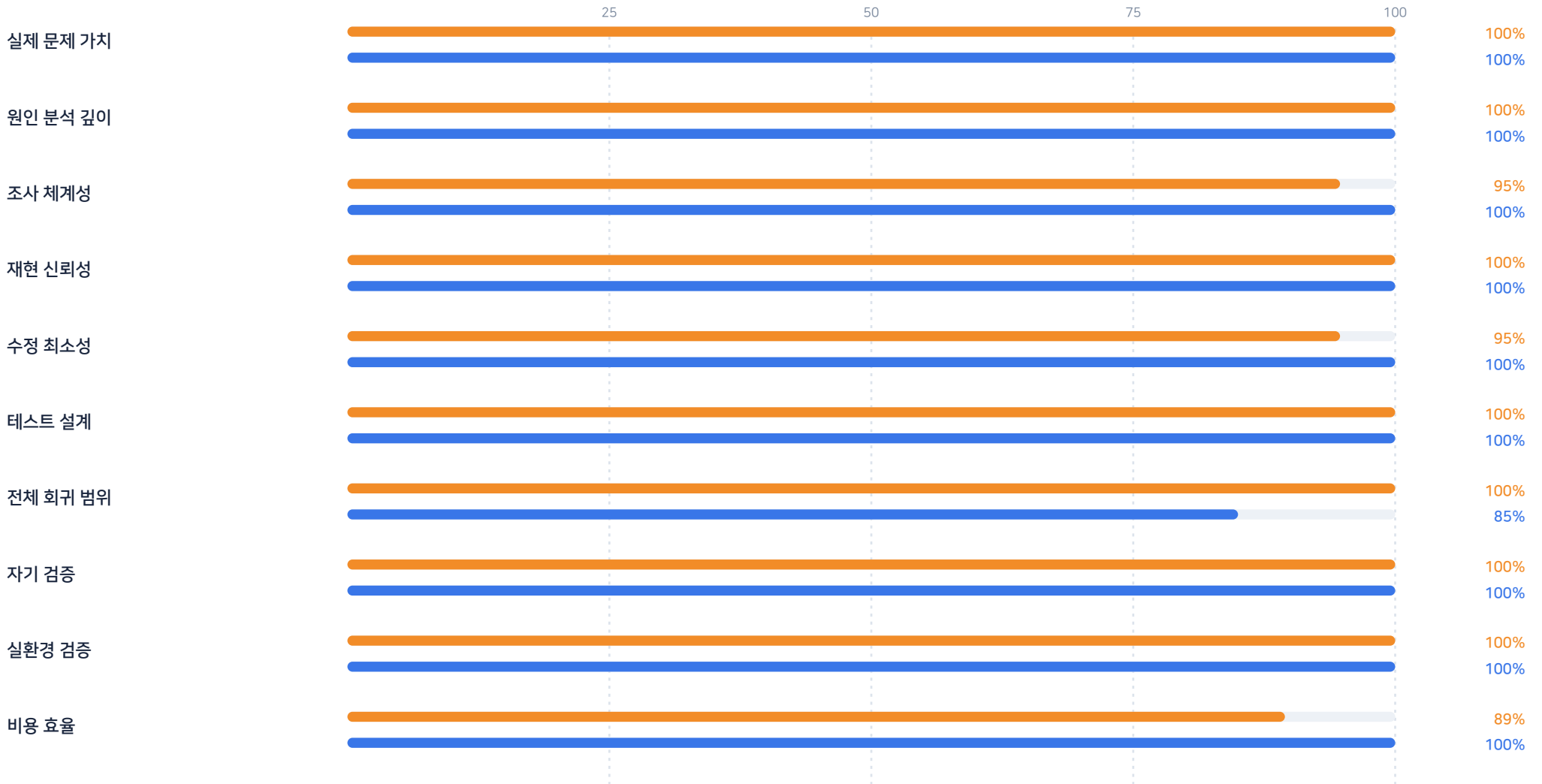
벤치마크 상대 백분위

이번 2회 실행 결과를 비교하기 위한 상대지수 · 100 = 두 실행 중 더 우수한 관측값 · 외부 모집단 백분위가 아님

Claude

Deel

100 = 항목별 최고 관측값



종합 상대지수

Claude 97.9%

Deel 98.5%

차이 2%p 이내 = 사실상 동급으로 해석

큰 차이 / 작은 차이

점수 차이가 실제 작업 과정에서 무엇을 의미하는지

큰 차이	전체 회귀 범위	Claude	50 + 11 + 18 + JS check까지 확인. Deel은 신규 6 + collaboration 50 + syntax 중심.
중간 차이	수정 최소성	Deel	receiveCollabState 한 경로에 기존 rebase 관례를 재사용해 더 국소적.
중간 차이	조사 체계성	Deel	서버 50개 기준선 → 미검증 클라이언트 → 호출부 비대칭 → 재현 순으로 좁힘.
작은 차이	테스트 신뢰성	동급	둘 다 수정 전 실패와 수정 후 성공, 역검증/변이 검증으로 테스트 유효성 확인.
차이 없음	실환경 검증	동급	둘 다 실제 브라우저 2인 E2E, 실제 SSE/캐럿 전체 검증은 남아 있음.

시간과 비용

속도 차이는 미미하지만 관측 비용 차이는 상대적으로 큼

비용



Deel이 \$1.33, 약 16.8% 낮음. 단 Claude는 token-saver가 적용된 조건이므로 원인 분리는 불가.

시간



차이는 61초, 약 2.7%. 실무적으로는 사실상 같은 작업 시간대로 보는 편이 타당.

검증 범위 비교

테스트 개수보다 무엇을 검증했는지와 실패를 실제로 잡는지가 중요

Claude CLI

신규 검사	신규 7개
검사 초점	IME 보류·base·서버 반영·연속 원격 변경까지 세분화
회귀 범위	기존 회귀 50 + 11 + 18
테스트 유효성	수정 일부를 되돌려 검사 07 실패 확인

Deel CLI

신규 검사	신규 6개
검사 초점	await 중 입력·재전송·base·동일 필드 충돌 검증
회귀 범위	기존 collaboration 50 + JS syntax
테스트 유효성	충돌 방어 제거 mutation에서 검사 실패 확인

최종 판단

이번 2회 결과에서 말할 수 있는 것과 아직 말하면 안 되는 것을 분리

현재 결과

품질은 거의 동급이다. Claude는 전체 회귀 검증 폭에서 우세하고, Deel은 조사 체계성·수정 국소성·관측 비용 효율에서 약간 우세했다. 시간은 사실상 동일했다.

말할 수 있음

두 실행 모두 실제 데이터 손실 버그를 재현·수정했다.
Deel 관측 비용이 이번 실행에서 16.8% 낮았다.
Claude의 회귀 검사 범위가 더 넓었다.

아직 단정하면 안 됨

Deel이 일반적으로 16.8% 더 싸다는 결론
token-saver 효과와 CLI 자체 효율의 분리
어느 도구가 항상 더 높은 품질이라는 결론

다음 벤치마크 권장

동일 유형 작업 5~10회 × Claude 순정 / Claude+token-saver / Deel 3조건으로 반복하고, 비용·시간·발견 결함 심각도·수정 LOC·재현 성공·회귀 범위·재작업 횟수를 함께 기록하면 도구 자체 효율을 훨씬 명확하게 비교할 수 있다.